KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade
Anand Nagar, Krishnankoil, Srivilliputtur (Via), Virudhunagar (Dt) - 626126, Tamil Nadu | info@kalasalingam.ac.in | www.kalasalingam.ac.in

MERN FULLSTACK REPORT

TITLE : MediCare360 – Hospital Management Platform

TEAM NAME : PIRATES

TEAM NUMBER : 11

Name of Student	**Register Number**
Patan Hussain Ali	99230040717
Parepalli Laxmi Sri Venkata Pradeep	99230040713
Mallikarjuna Rao Mulakala	99230040137
Nallajarla Venkata Gopinadh	99230041033
Nanduri Naga Venkata Surya Prakash	99230040693
Mamidikayala Saimourya	99230040651

Guide By

-Ajusha

ABSTRACT

MediCare-360 is a web-based **Hospital Management System** developed to simplify and organize hospital operations through a centralized digital platform. The main objective of this project is to improve the management of patients, doctors, hospital staff, and administrative activities while reducing manual paperwork and operational difficulties.

The system provides different user roles, including **Administrator, Doctor, Nurse, Receptionist, Pharmacist, Laboratory Technician, and Patient**. Patients can register through a separate registration facility, while administrators can create and manage staff accounts through the Admin Portal. Each user receives access according to their assigned role and permissions.

The application provides centralized user management, patient information management, staff management, department assignment, and role-based authentication. It is designed to improve communication and coordination between different hospital departments. The administrator can manage staff accounts and monitor hospital activities, while authorized staff members can access their respective modules.

MongoDB Atlas is used as the centralized database for storing persistent application information. The frontend communicates with backend APIs, which handle authentication, validation, business logic, and database operations. Security mechanisms such as password hashing and role-based access control help protect user accounts and application resources.

Overall, **MediCare-360** aims to provide an organized, secure, and user-friendly **hospital management platform** that improves administrative efficiency, data management, and coordination between hospital departments.

TABLE OF CONTENTS

TITLE		PAGE NO.
ABSTRACT		**ii**
LIST OF TABLES		**V**
LIST OF FIGURES		**Vi**
CHAPTER 1	**INTRODUCTION**	**1**
1.1	Introduction	
1.2	Problem Statement	
1.3	Objectives of the Project	
1.4	Scope of the Project	
CHAPTER 2	**SYSTEM ANALYSIS**	**4**
2.1	Introduction	
2.2	Analysis of the Existing System	
2.3	Limitations of the Existing System	
2.4	Proposed System	
2.5	Features of the Proposed System	
2.6	Functional Requirements	
2.7	Non-Functional Requirements	
2.8	Hardware Requirements	
2.9	Software Requirements	
2.10	Feasibility Study	
2.11	Summary	
CHAPTER 3	**SYSTEM DESIGN**	**12**
3.1	Introduction	
3.2	System Architecture	
3.3	Data Flow Design	
3.4	Database Design	

3.5	Module Design	
3.6	Security Design	
3.7	Summary	
CHAPTER 4	**SYSTEM IMPLEMENTATION**	**18**
4.1	Introduction	
4.2	Frontend Implementation	
4.3	Backend Implementation	
4.4	Database Implementation	
4.5	Authentication Implementation	
4.6	Appointment Booking Implementation	
4.7	Payment Implementation	
4.8	Doctor Dashboard Implementation	
4.9	Summary	
CHAPTER 5	**SYSTEM TESTING**	**22**
5.1	Introduction	
5.2	Types of Testing	
5.3	Test Cases	
5.4	Testing Result	
5.5	Summary	
CHAPTER 6	**RESULTS AND SCREENSHOTS**	**25**
6.1	Introduction	
6.2	Administrator Module Results	
6.3	Patient Module Results	
6.4	Doctor Module Results	
6.5	Nurse Module Results	

6.6	Receptionist Module Results	
6.7	Pharmacist Module Results	
6.8	Laboratory Technician Module Results	
6.9	Overall System Result	
CHAPTER 7	**CONCLUSION AND FUTURE ENHANCEMENT**	**36**
7.1	Conclusion	
7.2	Future Enhancements	
7.3	Final Summary	
CHAPTER 8	**REFERENCES**	**38**
8.1	References	
8.2	Official reference links	
8.3	References relate to MediCare-360	

LIST OF TABLES

TABLES	DETAILS	PAGE NO.
Table 1	Hardware requirements	10
Table 2	Technical Feasibility	10
Table 3	Test cases	23
Table 4	Official reference links	38
Table 5	References relate to MediCare-360	38

LIST OF FIGURES

FIGURES	DETAILS	PAGE NO.
Figure 1	System Architecture	13
Figure 2	Medicare-360 Home Page	25
Figure 3	Admin Login Page	26
Figure 4	Admin Dashboard	26
Figure 5	Create New User Account Page	26
Figure 6	Billings and invoices Page	27
Figure 7	Patient Registration Page	27
Figure 8	Patient Login Page	28
Figure 9	Patient Dashboard	28
Figure 10	Patient Book Doctor Appointment	28
Figure 11	Patient Bills & Payments	29
Figure 12	Doctor Dashboard	29
Figure 13	Issue prescriptions and Assign Patient to Lab	30
Figure 14	Doctor Schedule & Availability Management	30
Figure 15	Nurse Dashboard	31
Figure 16	Assigned Patients & Vitals Log and Patient Vitals & Ward Monitoring	31
Figure 17	Receptionist Dashboard	32
Figure 18	Receptionist Book Walk-in Patient Appointment	32
Figure 19	Pharmacist Dashboard	33
Figure 20	Pharmacy Medicine Inventory	33
Figure 21	Prescription Dispensing Desk and Dispense / Billing	33
Figure 22	Laboratory Technician Dashboard	34
Figure 23	Laboratory Technician in Patient Samples Reference	35

CHAPTER 1 – INTRODUCTION

1.1 Introduction

MediCare-360 is a web-based Hospital Management System developed to simplify and improve the management of hospital activities. Hospitals handle various operations, including patient registration, staff management, doctor information, department administration, and healthcare-related records. Managing these activities manually can be time-consuming and may result in errors, delays, and difficulties in maintaining accurate information.

To overcome these challenges, MediCare-360 provides a centralized digital platform that helps organize hospital operations. The system brings different hospital users together through separate interfaces and role-based access. It provides dedicated access for administrators, doctors, nurses, receptionists, pharmacists, laboratory technicians, and patients according to their responsibilities.

The Administrator has the authority to create and manage staff accounts, assign roles, manage departments, and monitor relevant hospital information. Patients can register through a separate registration facility, while staff members can access the system using credentials created by the administrator.

The application uses a frontend and backend architecture connected to a centralized MongoDB database. The frontend provides an interactive user interface, while the backend handles API requests, authentication, validation, and business logic. MongoDB Atlas is used to maintain persistent application records.

The main objective of MediCare-360 is to improve hospital management by reducing manual work, organizing information, providing secure user access, and supporting better coordination between hospital departments.

1.2 Problem Statement

Many hospitals manage their daily administrative and operational activities using manual procedures or separate software systems. These approaches can create difficulties in maintaining patient information, managing staff accounts, organizing departments, and accessing relevant hospital records.

The major problems include:

1. Manual patient registration and record maintenance can consume considerable time.
2. Managing different hospital staff accounts can become complicated.
3. Lack of centralized information makes data retrieval difficult.
4. Unauthorized access may occur when proper user access restrictions are not implemented.
5. Managing different departments and staff assignments can create confusion.
6. Maintaining accurate and updated hospital records can be challenging.
7. Communication between hospital departments may be inefficient.

8. Manual data entry can lead to duplicate records and human errors.
9. Administrators may face difficulties monitoring hospital activities.
10. Separate data storage methods can make information management unreliable.

MediCare-360 is developed to address these problems through an integrated web-based hospital management platform.

1.3 Objectives of the Project

The primary objective of MediCare-360 is to develop a centralized web-based platform that simplifies hospital administration and improves the management of patient and staff information.

The specific objectives are:

1. To develop a user-friendly hospital management application.
2. To provide separate registration facilities for patients.
3. To allow administrators to create and manage hospital staff accounts.
4. To implement role-based authentication and access control.
5. To provide dedicated interfaces for different hospital staff roles.
6. To organize patient and staff information in a centralized database.
7. To provide department and role-specific staff assignment facilities.
8. To improve the accessibility and management of hospital information.
9. To reduce manual paperwork and repetitive administrative activities.
10. To maintain persistent application data using MongoDB Atlas.
11. To improve data security through secure authentication and authorization.
12. To provide a maintainable and scalable application architecture.

1.4 Scope of the Project

The scope of MediCare-360 is to provide a centralized digital platform for managing hospital-related information and administrative activities.

The major modules include:

1. Administrator Module

- Administrator login and dashboard.
- Creation and management of staff accounts.
- Assignment of roles and appropriate departments.
- Monitoring of user information.
- Management of hospital activities and relevant records.

2. Patient Module

- Separate patient registration and login.
- Patient profile management.
- Access to authorized patient-related information.
- Organized storage of patient account details.

3. Doctor Module

- Dedicated doctor login.
- Doctor profile and department information.
- Access to authorized patient and clinical information.
- Doctor-specific dashboard and relevant workflows.

4. Nurse Module

- Nurse account management.
- Access to relevant patient-care information.
- Role-specific nursing workflows.

5. Receptionist Module

- Receptionist account management.
- Access to relevant registration and administrative workflows.
- Support for patient-related administrative activities.

6. Pharmacist Module

- Pharmacist account management.
- Access to pharmacy-related information and workflows, where implemented.

7. Laboratory Technician Module

- Laboratory staff account management.
- Access to laboratory-related information and workflows, where implemented.

8. Database Management

- Centralized storage using MongoDB Atlas.
- Persistent storage of user and application records.
- Retrieval and updating of records through backend APIs.
- Organized management of application data.

The system focuses on improving hospital administration, secure access, data organization, and coordination between different hospital roles.

CHAPTER 2 – SYSTEM ANALYSIS

MediCare-360 is a web-based Hospital Management System that centralizes hospital data and reduces manual work. It has separate portals for administrators, patients, and staff. The administrator creates staff accounts, assigns roles and departments, and monitors application data. Patients register themselves, book appointments, and manage their profiles. Staff log in with administrator-issued credentials and use only their authorized modules. The frontend is hosted on Netlify, the backend APIs on Render, and data is stored in MongoDB Atlas. The APIs handle authentication, validation, and database operations. Role-based access control restricts each user to permitted features. The system improves data organization, strengthens security, and streamlines hospital operations.

2.1 Introduction

System analysis is an important phase in software development that identifies the requirements, functionalities, resources, and limitations of an application. It helps understand the problems in the existing system and determines how the proposed application can address them.

MediCare-360 is designed to simplify hospital management by providing a centralized platform for administrators, doctors, nurses, receptionists, pharmacists, laboratory technicians, and patients.

The system analysis identifies the functional requirements, non-functional requirements, hardware requirements, software requirements, and feasibility of the proposed application. It also explains how the different modules work together to improve hospital operations.

2.2 Analysis of the Existing System

In traditional hospital management systems, many administrative and patient-related activities are performed manually or through separate software applications. Patient information, staff details, department information, and other hospital records may be maintained in different locations.

This approach can create difficulties in accessing updated information and coordinating activities between hospital departments.

The existing approach may involve the following activities:

- Patients provide their details manually during registration.
- Hospital staff maintain patient and administrative records.
- Administrators manage staff information through separate procedures.
- Staff assignments may be maintained manually.
- Retrieving patient or staff information can take additional time.
- User access may not be consistently controlled according to job responsibilities.
- Updating records across multiple systems can be difficult.
- Hospital administrators may have limited centralized monitoring facilities.

These limitations can increase administrative workload and reduce the efficiency of hospital operations.

2.3 Limitations of the Existing System

The traditional hospital management approach has several limitations:

1. Time Consumption: Manual registration and record management require additional time and effort.

2. Manual Record Management: Maintaining patient and staff information manually may result in errors and duplicate records.

3. Lack of Centralized Information: Information stored in separate systems can be difficult to access and manage.

4. Staff Management Difficulties: Creating, updating, and assigning staff accounts through manual procedures can become complicated.

5. Security Issues: Without proper authentication and authorization, sensitive information may be accessible to unauthorized users.

6. Department Assignment Problems: Incorrect staff assignments can create confusion in hospital operations.

7. Data Maintenance Difficulties: Maintaining accurate and updated records can be challenging.

8. Administrative Workload: Administrators may need to perform repetitive tasks related to account creation and information management.

9. Limited Accessibility: Authorized users may face difficulties accessing required information when records are maintained separately.

10. Poor Coordination: Separate workflows can affect communication and coordination between hospital departments.

2.4 Proposed System

The proposed MediCare-360 system is a web-based Hospital Management System designed to centralize hospital information and simplify administrative operations.

The system provides separate access for administrators, patients, and different hospital staff roles. The administrator can create staff accounts, assign appropriate roles, manage department assignments, and monitor relevant application information.

Patients are provided with a separate registration facility. Staff members use administrator-created credentials to access their respective dashboards and authorized modules.

The application uses MongoDB Atlas as its centralized database for persistent information storage. Backend APIs manage data communication, authentication, validation, and database operations. The frontend provides a user-friendly interface for accessing the application's features.

Role-based access control ensures that users can access only the features and information permitted for their assigned roles.

The proposed system aims to reduce manual work, improve data organization, strengthen access control, and provide an efficient environment for managing hospital activities.

2.5 Features of the Proposed System

The MediCare-360 system provides the following major features:

1. Centralized Admin Management

- Administrator dashboard.
- Staff account creation and management.
- Role assignment and department management.
- Monitoring of relevant hospital information.

2. Patient Management

- Separate patient registration.
- Patient login and profile management.
- Organized patient account information.
- Access to authorized patient-related services.

3. Staff Management

- Creation of staff accounts by the administrator.
- Support for Doctor, Nurse, Receptionist, Pharmacist, and Laboratory Technician roles.
- Role-specific account management.
- Staff profile management.

4. Role-Based Authentication

- Separate access according to user roles.
- Secure login and logout.
- Password hashing.
- Protected backend APIs.
- Role-based authorization.

5. Department and Designation Management

- Appropriate department assignments for staff roles.
- Doctor-specific medical specialties.

- Nursing, pharmacy, laboratory, and administrative assignments.
- Role-specific validation of department and designation information.

6. Centralized Database Management

- MongoDB Atlas integration.
- Persistent user and application records.
- Database-backed create, read, update, and delete operations.
- Reduced dependence on hardcoded records.

7. Dashboard Management

- Role-specific dashboard interfaces.
- Organized presentation of relevant information.
- Database-backed statistics where implemented.
- Appropriate empty states when no records are available.

8. User-Friendly Interface

- Organized navigation.
- Responsive interface.
- Clear forms and validation messages.
- Consistent visual design.

2.6 Functional Requirements

Functional requirements describe the operations and services that the system must provide to its users.

The functional requirements of MediCare-360 are classified according to the user roles.

2.6.1 Administrator Functional Requirements

- The system must provide a secure administrator login.
- The administrator must be able to access the admin dashboard.
- The administrator must be able to create staff accounts.
- The administrator must be able to assign appropriate roles.
- The administrator must be able to assign role-specific departments or designations.
- The administrator must be able to view and manage staff information.
- The administrator must be able to manage relevant patient records.
- The administrator must be able to monitor available hospital information.
- The system must save account information in MongoDB Atlas.

- The administrator must be able to manage account status according to permissions.

2.6.2 Patient Functional Requirements

- The system must allow new patients to register separately.
- Registered patients must be able to log in using valid credentials.
- Patients must be able to access their authorized dashboard.
- Patients must be able to view and manage permitted profile information.
- Patient information must be stored in MongoDB.
- Patients must not be able to access restricted staff or administrator modules.

2.6.3 Doctor Functional Requirements

- The system must provide a separate doctor login.
- Doctors must be able to access their dedicated dashboard.
- Doctors must be able to view their authorized information.
- Doctor accounts must be created through the administrator.
- Doctors must receive appropriate medical department or specialty assignments.
- Doctors must not access unauthorized administrator functions.

2.6.4 Nurse Functional Requirements

- Nurses must be able to log in using administrator-created credentials.
- Nurses must access their dedicated authorized functions.
- The system must support nursing-related department assignments.
- Nurse account information must be maintained in MongoDB.

2.6.5 Receptionist Functional Requirements

- Receptionists must be able to log in using their assigned credentials.
- The system must provide access to relevant administrative workflows.
- Receptionists must receive appropriate service-area assignments.
- Receptionist information must be stored and retrieved from MongoDB.

2.6.6 Pharmacist Functional Requirements

- Pharmacists must be able to access their authorized dashboard.
- Pharmacist accounts must be created and managed by the administrator.
- The system must support pharmacy-related assignments.
- Pharmacist information must be stored in the database.

2.6.7 Laboratory Technician Functional Requirements

- Laboratory technicians must be able to log in using their assigned credentials.
- The system must provide access to authorized laboratory-related functions.
- Laboratory staff must receive appropriate laboratory assignments.
- Laboratory account information must be stored in MongoDB.

2.7 Non-Functional Requirements

Non-functional requirements describe the quality attributes and performance expectations of the application.

1. Security

The system uses authentication and role-based authorization to protect application resources. Passwords must be securely hashed, and unauthorized access to restricted modules must be prevented.

2. Usability

The application provides an organized and user-friendly interface that allows administrators, staff, and patients to access their respective features easily.

3. Performance

The system should process user requests, retrieve database information, and display relevant dashboard data efficiently.

4. Reliability

The application should maintain accurate and persistent records through MongoDB Atlas and handle errors appropriately.

5. Scalability

The system should support future expansion, including additional staff roles, hospital departments, and management modules.

6. Maintainability

The application should follow a modular structure that separates frontend components, backend services, authentication, and database operations.

7. Accessibility

The application should provide responsive layouts that support different screen sizes and devices.

8. Data Integrity

The system should validate user information, prevent duplicate accounts where applicable, and maintain consistency between related database records.

2.8 Hardware Requirements

The following hardware requirements are suitable for developing and running the MediCare-360 application.

Component	Minimum requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Hard Disk	10 GB free space
Display	Standard monitor
Input Devices	Keyboard and mouse
Network	Internet connection

Table 1 Hardware requirements

2.9 Software Requirements

The following technologies and software tools are used or required for the development and operation of MediCare-360. Confirm the final backend framework against the actual project implementation.

Software / Technology	Purpose
Software / Technology	**Purpose**
React.js	Frontend development
JavaScript	Application programming
HTML5	Web page structure
CSS3	Styling and responsive design
Python / FastAPI	Backend API development, if retained in the current implementation
MongoDB Atlas	Cloud database management
MongoDB Driver / ODM	Database communication
JWT	User authentication
Password Hashing Library	Secure password storage
Visual Studio Code	Development environment
Git and GitHub	Version control

Table 2 : **Software Requirements**

2.10 Feasibility Study

A feasibility study is conducted to determine whether the proposed system can be developed and operated successfully.

The MediCare-360 project is evaluated based on the following feasibility factors.

2.10.1 Technical Feasibility

The project is technically feasible because it uses established web development technologies and a centralized database architecture. The frontend provides an interactive interface, while backend APIs manage business logic, authentication, and database operations.

MongoDB Atlas provides cloud-based database storage for maintaining application records. These technologies support modular development and future improvements.

2.10.2 Economic Feasibility

The project is economically feasible because it uses widely available development tools and technologies. Development environments, programming frameworks, and database services offer accessible options for building and maintaining the application.

This reduces the need for expensive software and infrastructure during development.

2.10.3 Operational Feasibility

The system is operationally feasible because it provides separate interfaces for different hospital users. Administrators can manage staff accounts, while patients and staff members can access their respective modules.

The organized interface reduces the complexity of managing hospital information and requires only basic computer or smartphone knowledge.

2.10.4 Schedule Feasibility

The project can be developed using a modular implementation approach. Individual modules such as authentication, patient management, staff management, database integration, and dashboards can be developed and tested separately.

This approach helps organize development activities and supports completion within the available project schedule.

2.11 Summary

This chapter discussed the system analysis of MediCare-360, including the existing system, its limitations, proposed system, features, functional requirements, non-functional requirements, hardware requirements, software requirements, and feasibility study.

The analysis identifies the essential requirements for developing a centralized and organized Hospital Management System. It provides the foundation for the system design and implementation phases.

CHAPTER 3

SYSTEM DESIGN

3.1 Introduction

System design explains the overall architecture, database structure, modules, and workflow of the MediCare-360 application. It describes how administrators, patients, and hospital staff interact with the system.

The design focuses on separating the frontend, backend, authentication, and database operations. This modular architecture makes the application easier to maintain, test, and extend.

3.2 System Architecture

MediCare-360 follows a client-server architecture with a centralized database.

The major components are:

1. Frontend Layer

The frontend provides the user interface through which administrators, patients, and staff members interact with the application. It includes login pages, registration forms, dashboards, account management interfaces, and other available modules.

2. Backend Layer

The backend receives frontend requests, validates input, performs authentication and authorization, executes business logic, and communicates with MongoDB Atlas.

3. Database Layer

MongoDB Atlas stores persistent application information, including user accounts, patient records, staff information, department assignments, and other implemented hospital records.

4. Authentication Layer

The authentication system verifies user credentials and establishes authenticated access. Role-based authorization controls access to protected application features.

System Architecture Flow:

User → Frontend Interface → Backend API → Authentication and Validation → MongoDB Atlas → Backend Response → Frontend Dashboard.

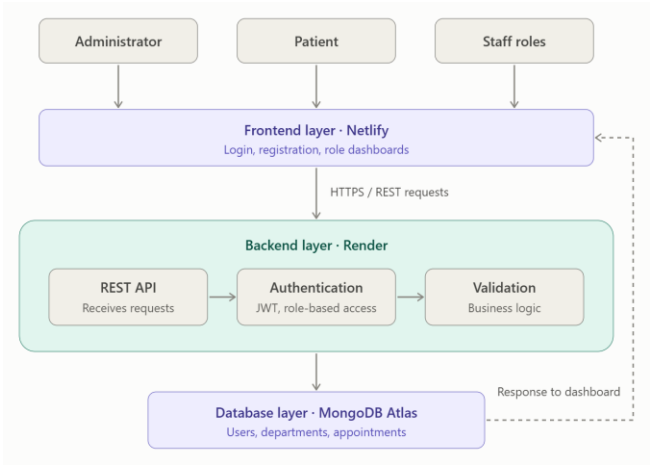


Fig 1 System Architecture

3.3 Data Flow Design

The MediCare-360 system acts as a centralized process connecting administrators, patients, and hospital staff.

Level 0 – Data Flow

- Administrator creates and manages staff accounts.
- Patients submit registration information.
- Staff members log in using assigned credentials.
- Authorized users request relevant hospital information.
- The backend processes requests and communicates with MongoDB Atlas.
- The system returns authorized information to the frontend.

Level 1 – Data Flow

1. A user opens the MediCare-360 application.
2. The user selects the appropriate login or registration facility.
3. The frontend sends the entered information to the backend.
4. The backend validates the submitted information.
5. The authentication process verifies the user's credentials.
6. The system identifies the user's authorized role.
7. The backend retrieves or updates the required records in MongoDB Atlas.
8. The frontend displays the appropriate dashboard and information.
9. Authorized changes are saved to the database.
10. The user can log out securely.

3.4 Database Design

MediCare-360 uses MongoDB Atlas as its centralized database for storing persistent application information.

The database is designed to organize information into collections according to the application's modules.

The possible collection structure includes:

1. Users Collection

Stores user account information, such as:

- User ID.
- Full name.
- Email address.
- Password hash.
- Assigned role.
- Phone number.
- Account status.
- Created and updated timestamps.

2. Patients Collection

Stores patient-related information, such as:

- Patient ID.
- Name.
- Contact information.
- Profile details.
- Relevant patient information.

3. Staff Collection

Stores hospital staff information, such as:

- Staff ID.
- Name.
- Email address.
- Role.
- Department assignment.
- Designation or specialty, where applicable.

- Account status.

4. Departments Collection

Stores department information, such as:

- Department ID.
- Department name.
- Department code.
- Department category.
- Status.

5. Other Application Collections

Additional collections may be used for appointments, prescriptions, laboratory records, pharmacy inventory, billing, or other modules if those features are implemented in the application.

MongoDB document relationships should use appropriate document identifiers and references. The actual collection names and fields must follow the implemented database models.

All persistent records must be stored in MongoDB Atlas rather than hardcoded in backend or frontend source files.

3.5 Module Design

The MediCare-360 application is divided into the following modules.

1. Administrator Module

- Administrator authentication.
- Admin dashboard.
- Staff account creation.
- Role assignment.
- Department management.
- User account monitoring.

2. Patient Module

- Patient registration.
- Patient login.
- Patient profile management.
- Access to authorized patient features.

3. Doctor Module

- Doctor authentication.

- Doctor dashboard.
- Doctor profile and specialty information.
- Access to authorized clinical information.

4. Nurse Module

- Nurse authentication.
- Nursing-related dashboard.
- Access to authorized patient-care information.

5. Receptionist Module

- Receptionist authentication.
- Access to administrative and registration workflows.
- Relevant patient-related information management.

6. Pharmacist Module

- Pharmacist authentication.
- Pharmacy-related dashboard and authorized functions.

7. Laboratory Technician Module

- Laboratory staff authentication.
- Laboratory-related dashboard and authorized functions.

8. Database Module

- MongoDB Atlas connectivity.
- Persistent data storage.
- Data retrieval and updating.
- Database validation and error handling.

3.6 Security Design

The system uses security mechanisms to protect user accounts and application information.

1. Authentication

The system verifies user credentials before providing access to protected resources.

2. Password Hashing

Passwords are securely hashed before being stored in the database. Plaintext passwords must never be saved in MongoDB.

3. Role-Based Access Control

Access is restricted according to the user's assigned role. Administrators, doctors, nurses, receptionists, pharmacists, laboratory technicians, and patients receive appropriate permissions.

4. Protected APIs

Backend API endpoints require appropriate authentication and authorization.

5. Database Security

MongoDB Atlas credentials are maintained through private environment configuration. Database access is restricted using appropriate permissions.

6. Input Validation

User-submitted information is validated to prevent invalid or inconsistent records.

7. Data Privacy

Users should only be able to retrieve information that they are authorized to access.

3.7 Summary

This chapter explained the system architecture, data flow, database design, module structure, and security design of MediCare-360.

The design provides a structured foundation for developing a centralized hospital management platform with separate user roles and persistent database storage.

CHAPTER 4 – SYSTEM IMPLEMENTATION

4.1 Introduction

System implementation is the process of developing the MediCare-360 application using frontend, backend, and database technologies.

The application provides separate interfaces for administrators, patients, and hospital staff. It integrates user authentication, account management, role assignment, and database operations to support organized hospital administration.

The implementation follows a modular approach in which the frontend, backend, authentication, and database components work together.

4.2 Frontend Implementation

The frontend is developed using modern web technologies to provide an interactive and user-friendly interface.

The major frontend components include:

- Home page and application navigation.
- Administrator login page.
- Patient registration and login pages.
- Staff login interface.
- Admin dashboard.
- Create and manage user account forms.
- Role selection and department assignment.
- Doctor, nurse, receptionist, pharmacist, and laboratory staff interfaces.
- User profile and account management pages.
- Loading indicators and form validation messages.

The frontend communicates with backend APIs to retrieve and update application information.

The interface is designed to provide consistent navigation, organized forms, and role-specific access.

4.3 Backend Implementation

The backend handles application requests, business logic, authentication, validation, and database communication.

The major backend operations include:

- User registration and login.
- Staff account creation by administrators.

- Role and department validation.
- Authentication and authorization.
- User profile retrieval and updates.
- Database read and write operations.
- API error handling.
- Protected access to restricted resources.

The backend acts as the communication layer between the frontend and MongoDB Atlas.

It ensures that submitted data is validated before being stored and that users can access only authorized resources.

4.4 Database Implementation

MongoDB Atlas is used as the centralized database for storing MediCare-360 application information.

The backend connects to MongoDB Atlas using a secure database connection configured through environment variables.

The database maintains persistent information, including:

- User accounts.
- Patient information.
- Staff profiles.
- Role assignments.
- Department information.
- Account status.
- Other records belonging to implemented modules.

Database operations are performed through the backend. The frontend does not connect directly to MongoDB Atlas.

The application uses database operations to create, retrieve, update, and delete records according to user permissions.

This approach ensures that application data remains available after browser refreshes and backend restarts.

4.5 Authentication Implementation

The authentication module verifies the identity of users before granting access to the application.

MediCare-360 supports separate patient registration and administrator-controlled staff account creation.

The authentication process follows these steps:

1. The user enters the registered email address and password.
2. The frontend sends the credentials to the backend.
3. The backend retrieves the corresponding account from MongoDB.
4. The submitted password is verified against the stored password hash.
5. The system checks the account status and assigned role.
6. The backend establishes an authenticated session or issues an authentication token according to the implemented architecture.
7. The user is redirected to the appropriate dashboard.
8. Protected API requests are checked for authentication and authorization.

The system must prevent users from accessing dashboards or information outside their assigned permissions.

4.6 Appointment Booking Implementation

The staff account management module allows the administrator to create and manage hospital staff accounts.

The administrator selects the appropriate role, enters the staff member's details, assigns a valid department or designation, and creates the account.

The supported staff roles include:

- Doctor.
- Nurse.
- Receptionist.
- Pharmacist.
- Laboratory Technician.
- Administrator.

The system applies role-specific assignment options. For example, doctors can receive medical specialties, nurses can receive nursing units, receptionists can receive administrative service areas, pharmacists can receive pharmacy assignments, and laboratory technicians can receive laboratory sections.

The backend validates the selected role and assignment before saving the account to MongoDB Atlas.

This module helps organize staff information and provides controlled account creation.

4.7 Patient Management Implementation

The patient management module provides a separate registration facility for patients.

New patients can enter their required information and create an account. The backend validates the submitted details and stores the account in MongoDB Atlas.

Registered patients can authenticate using their credentials and access their authorized features.

The separate registration process ensures that patient onboarding is distinct from administrator-controlled staff account creation.

4.8 Dashboard Implementation

MediCare-360 provides role-specific dashboards for authorized users.

The administrator dashboard presents relevant hospital and user management information. Staff dashboards provide access to features associated with their assigned responsibilities.

Dashboard information should be retrieved from MongoDB through backend APIs rather than maintained as hardcoded records.

When no records are available, the application should display appropriate zero values or empty states instead of fabricated information.

4.9 Summary

The MediCare-360 system is implemented using a modular frontend, backend, and database architecture. It provides patient registration, staff account management, role-based authentication, department assignment, and centralized database operations.

The implementation focuses on simplifying hospital administration, maintaining organized records, and providing secure access to different application modules.

CHAPTER 5 – SYSTEM TESTING

5.1 Introduction

System testing is an important phase of software development used to verify whether the application works according to its requirements.

Testing helps identify errors, validate application functionality, and ensure proper communication between the frontend, backend, and database.

MediCare-360 requires testing of its authentication, patient registration, staff account creation, role assignment, database operations, and dashboard functionality.5.2 Types of Testing

5.2 Types of Testing

1. Unit Testing

Unit testing checks individual application components and functions, such as input validation, role checking, authentication functions, and database-related services.

2. Integration Testing

Integration testing verifies communication between the frontend, backend APIs, authentication module, and MongoDB Atlas.

3. Functional Testing

Functional testing checks whether the major application features work according to the specified requirements.

4. Validation Testing

Validation testing checks whether user-submitted information is accepted or rejected according to the defined rules.

5. Authentication Testing

Authentication testing verifies successful login, invalid credentials, account status, and role-specific access restrictions.

6. Database Testing

Database testing verifies whether records are correctly created, retrieved, updated, and deleted in MongoDB Atlas.

7. User Interface Testing

User interface testing checks page layouts, navigation, form behavior, dropdown options, and responsive design.

8. Security Testing

Security testing verifies that unauthorized users cannot access protected resources and that sensitive information is properly protected.

5.3 Test Cases

Test Case	Expected Result
Patient registration	Patient account should be created successfully
Patient login	Valid credentials should allow login
Invalid login	Appropriate error message should be displayed
Admin login	Administrator should access the admin dashboard
Staff account creation	Admin should be able to create staff accounts
Doctor account creation	Doctor account should be saved with the correct role
Nurse account creation	Nurse account should be saved with the correct role
Receptionist account creation	Receptionist account should be saved with the correct role
Pharmacist account creation	Pharmacist account should be saved with the correct role
Lab Technician account creation	Laboratory account should be saved with the correct role
Department assignment	Correct role-specific options should be displayed
Invalid role assignment	Backend should reject incompatible assignments
MongoDB storage	New records should appear in the correct collection
Record update	Updated information should persist in MongoDB
Account status	Inactive accounts should not be allowed to log in
Unauthorized access	Restricted API requests should be rejected
Data persistence	Records should remain available after a server restart

Table 3 Test Cases

5.4 Testing Result

The testing process is designed to verify the major functionalities of MediCare-360, including patient registration, staff account creation, role-based authentication, department assignment, and database integration.

The test results should be recorded after executing each test case in the application.

Successful tests should confirm that user information is saved in MongoDB Atlas, authenticated users receive the correct access, and invalid requests are handled appropriately.

Any errors identified during testing should be corrected and retested before final deployment.

5.5 Summary

This chapter discussed the testing methods and test cases used to evaluate MediCare-360.

Testing helps verify the functionality, security, reliability, and integration of the application's major modules. It also ensures that the system maintains accurate records and provides appropriate access to different user roles.

CHAPTER 6 – RESULTS AND SCREENSHOTS

6.1 Introduction

This chapter presents the results and screenshots of the MediCare-360 application. It explains the major modules and demonstrates how different users interact with the system.

The application provides a centralized interface for administrators, patients, and hospital staff. Each user can access the features permitted by their assigned role.

Screenshots captured from the actual application should be included in this chapter to demonstrate the implemented functionalities.

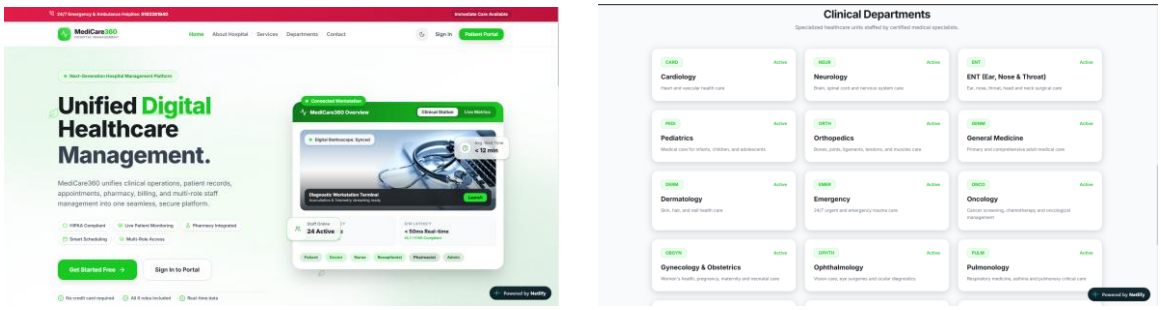


Fig 2: Home Page

6.2 Administrator Module Results

The Administrator Module provides centralized control over hospital user management and administrative activities.

The administrator can:

- Log in through the administrator login page.
- Access the admin dashboard.
- Create new staff accounts.
- Select appropriate staff roles.
- Assign role-specific departments or designations.
- View and manage user information.
- Monitor relevant hospital records.

The staff account creation facility allows administrators to manage different hospital roles through a common interface.

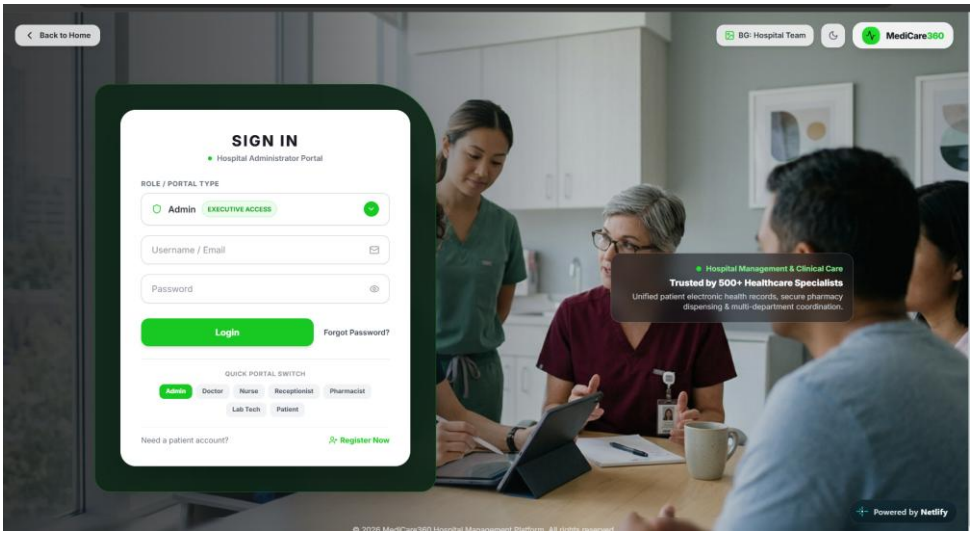


Figure 3: Admin Login Page

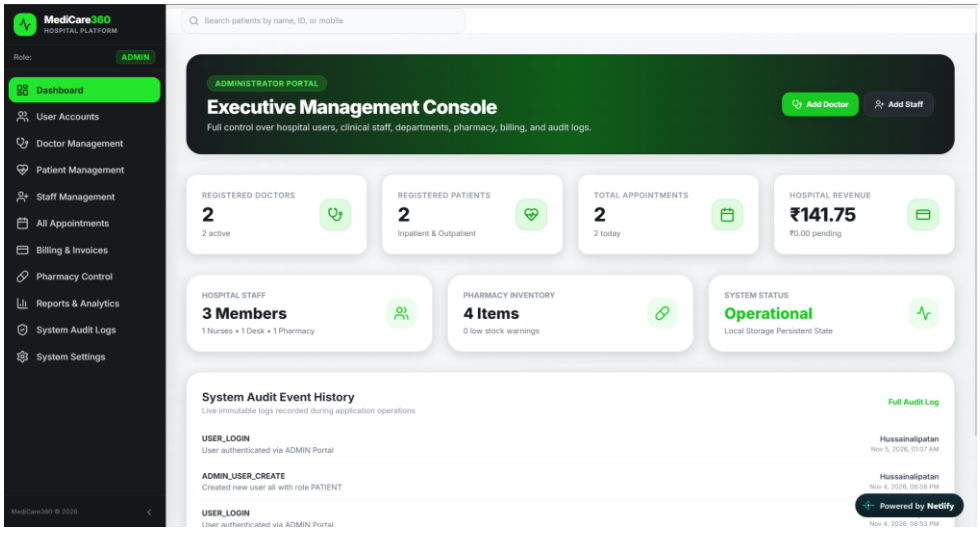


Figure 4: Admin Dashboard

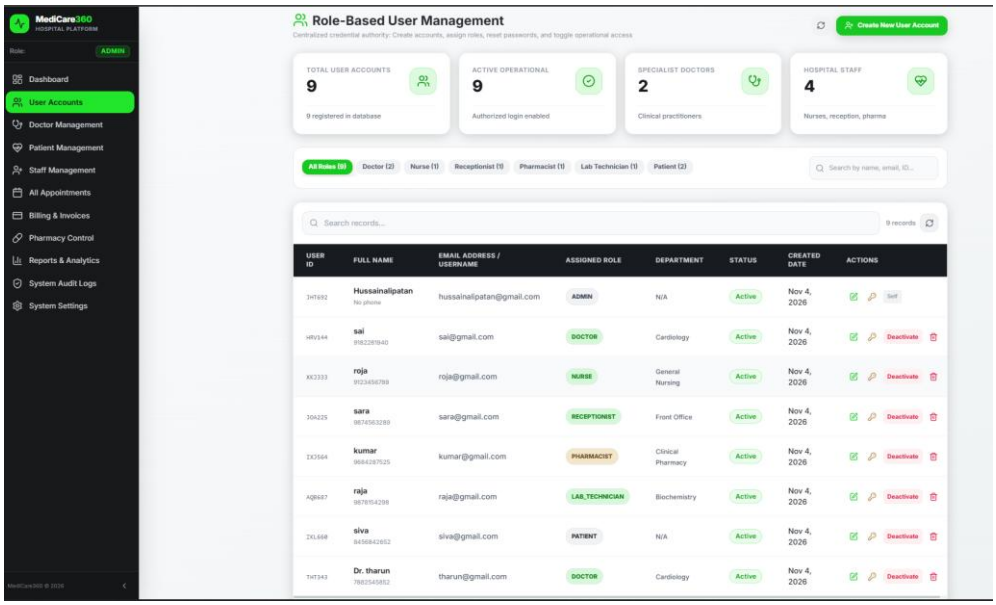


Figure 5: Create New User Account Page

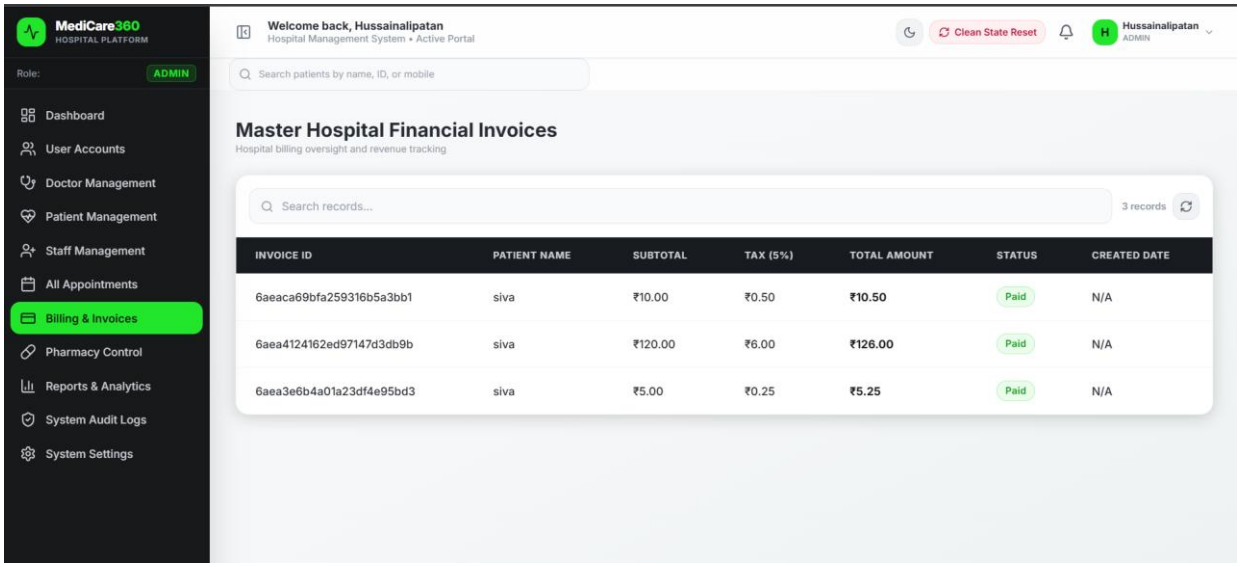


Figure 6: Billings and invoices Page

6.3 Patient Module Results

The Patient Module provides a separate registration and login facility.

Patients can:

- Register a new account.
- Log in using valid credentials.
- Access their authorized dashboard.
- View and manage available profile information.
- Access patient-related application features.

Patient account information is stored in MongoDB Atlas through backend operations.

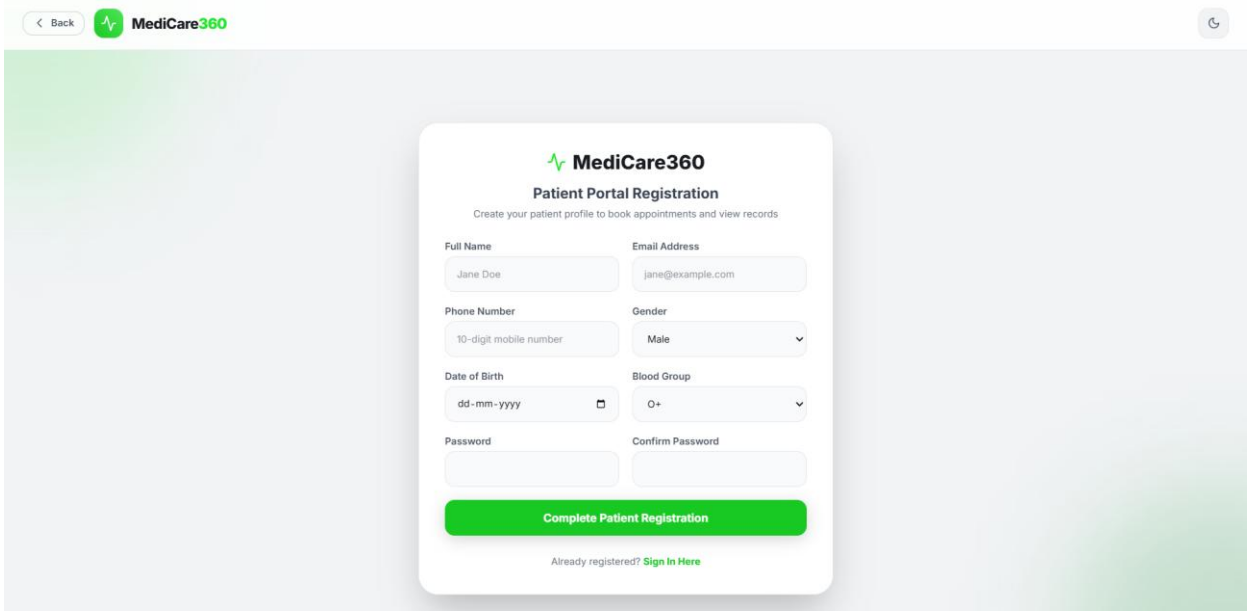


Figure 7 : Patient Registration Page



Figure 8: Patient Login Page

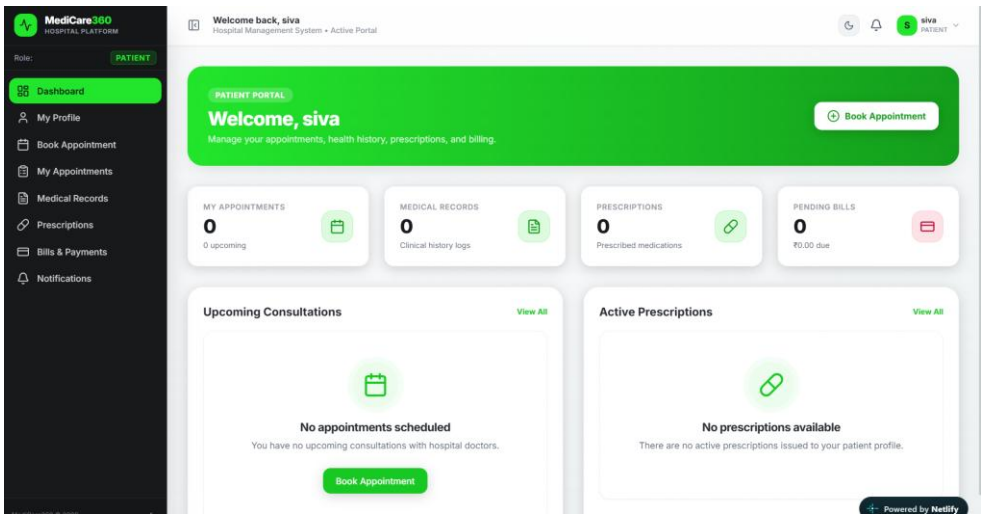


Figure 9: Patient Dashboard

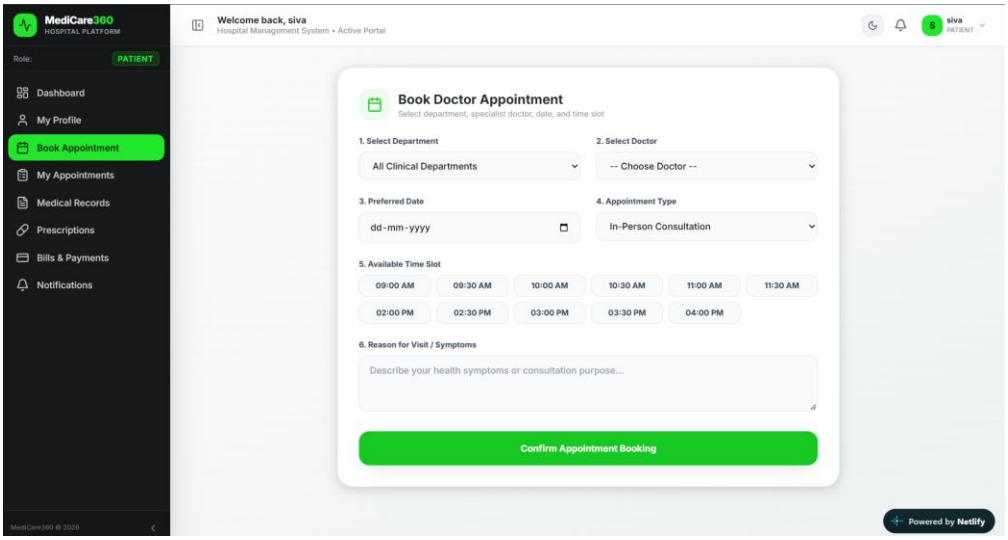


Figure 10: Patient Book Doctor Appointment

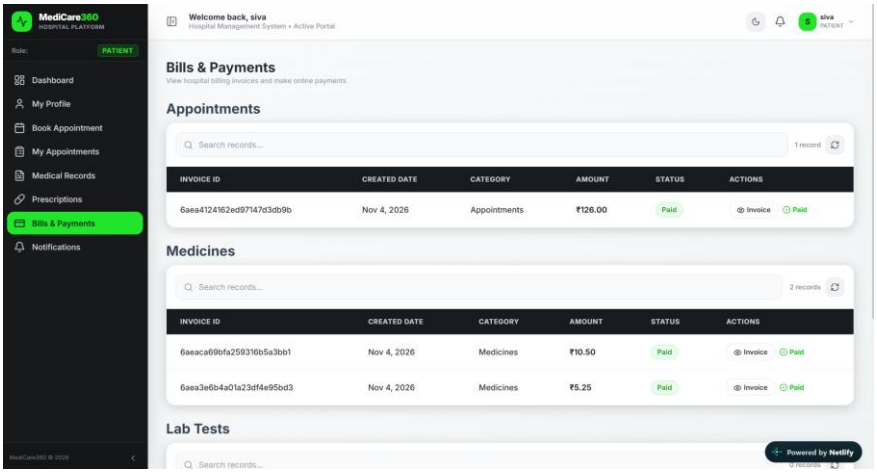


Figure 11: Patient Bills & Payments

6.4 Doctor Module Results

The Doctor Module provides a dedicated interface for doctors to access their hospital-related information and authorized clinical activities.

- Secure doctor login using administrator-created credentials.
- Doctor-specific dashboard.
- Doctor profile and personal information management.
- Medical department and specialty assignment.
- Access to authorized patient information.
- Support for doctor-related hospital activities.
- Role-based access to relevant application features.

The doctor dashboard helps doctors access their assigned information in an organized manner. The department assignment feature allows administrators to assign specialties suchas Cardiology, Neurology, Pediatrics, and other supported medical departments.

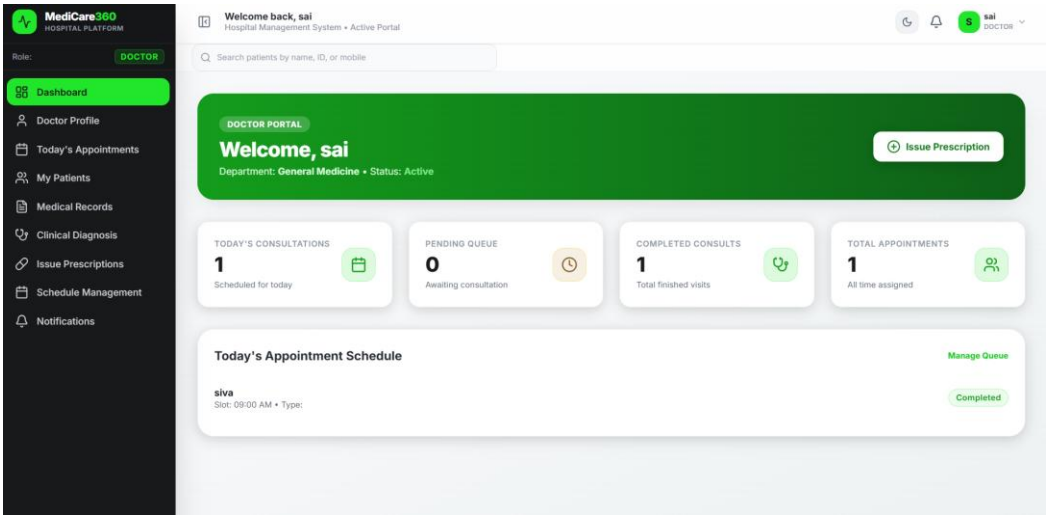


Figure 12: Doctor Dashboard

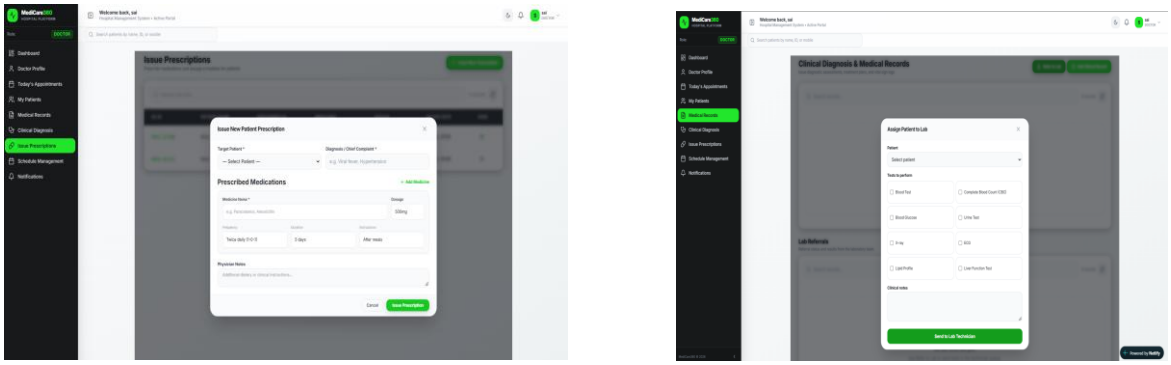


Figure 13: Issue prescriptions and Assign Patient to Lab

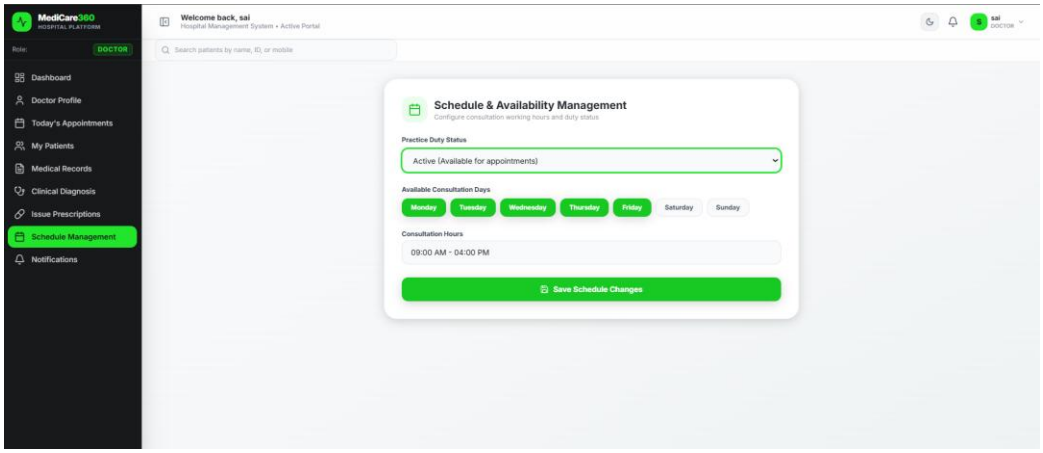


Figure 14: Doctor Schedule & Availability Management

6.5 Nurse Module Results

The Nurse Module is designed to provide nurses with access to their assigned hospital information and nursing-related activities.

- Secure nurse login.
- Nurse-specific dashboard.
- Nurse profile management.
- Nursing department and unit assignment.
- Access to authorized patient-care information.
- Role-based access to nursing-related features.
- Centralized storage of nurse account information.

Administrators can create nurse accounts and assign appropriate departments or units, such as General Nursing, ICU, Emergency, Pediatrics, and other supported nursing units.

.

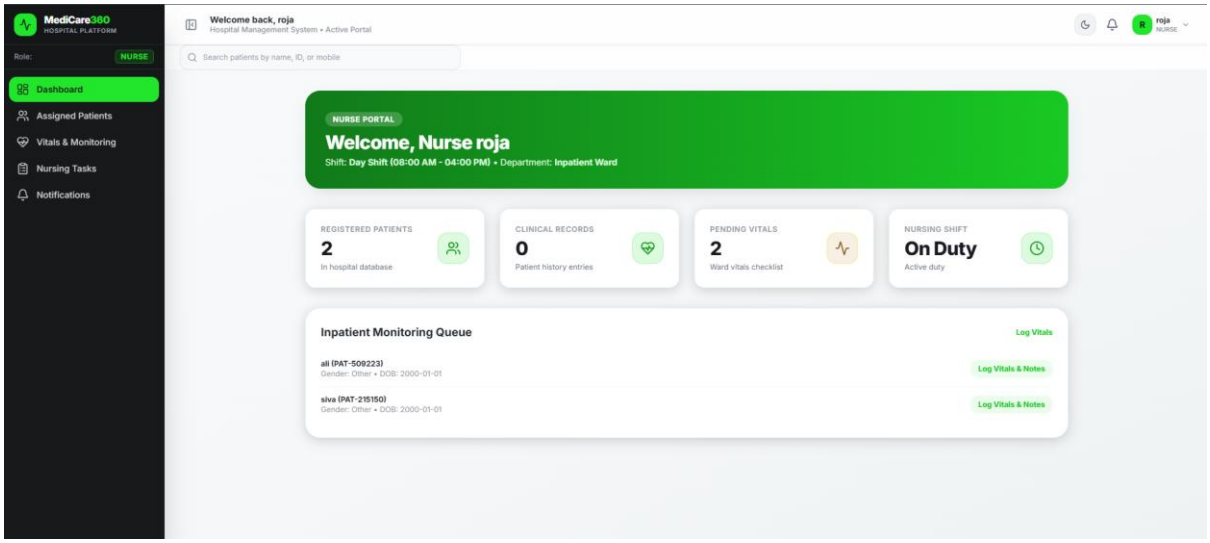


Figure 15: Nurse Dashboard

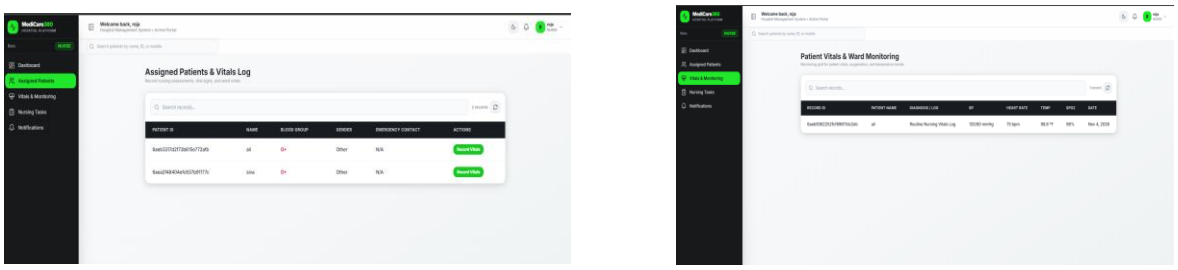


Figure 16: Assigned Patients & Vitals Log and Patient Vitals & Ward Monitoring

6.6 Receptionist Module Results

The Receptionist Module provides an interface for receptionists to access hospital registration and administrative workflows.

- Secure receptionist login.
- Receptionist-specific dashboard.
- Receptionist profile management.
- Front-office and administrative service assignments.
- Access to authorized patient registration information.
- Support for relevant appointment and administrative workflows.
- Role-based access to permitted features.

The administrator can create receptionist accounts and assign suitable service areas such as Front Office, Patient Registration, Appointment Desk, and Billing Desk, depending on the application's available configuration.

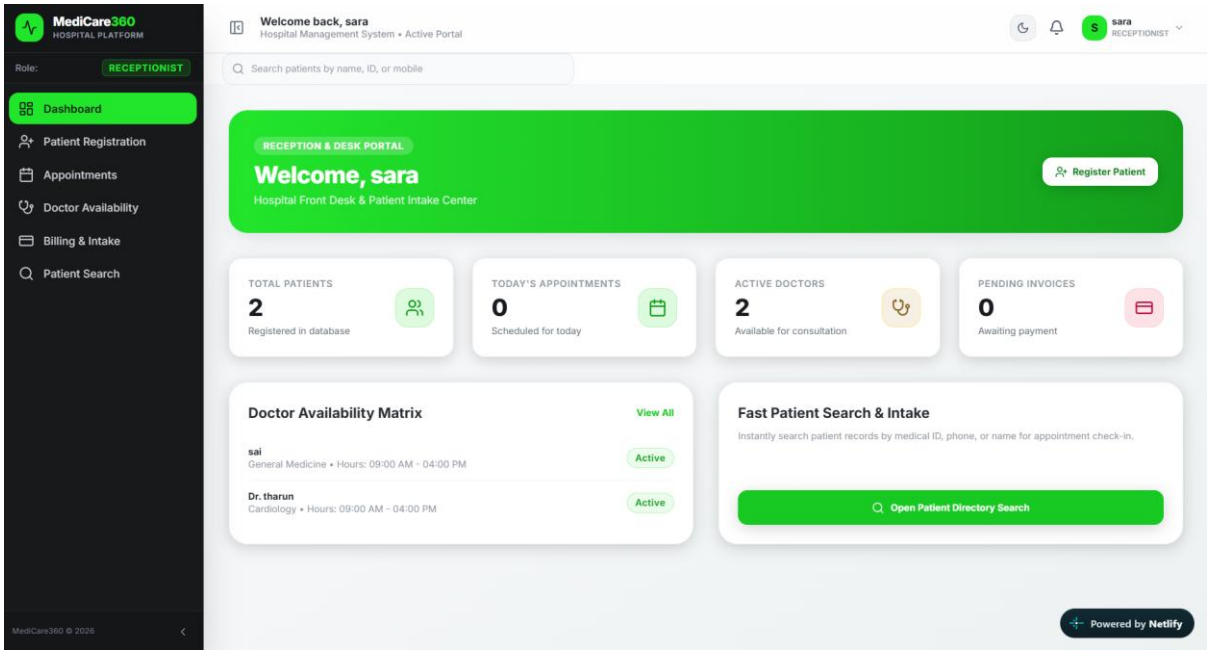


Figure 17: Receptionist Dashboard

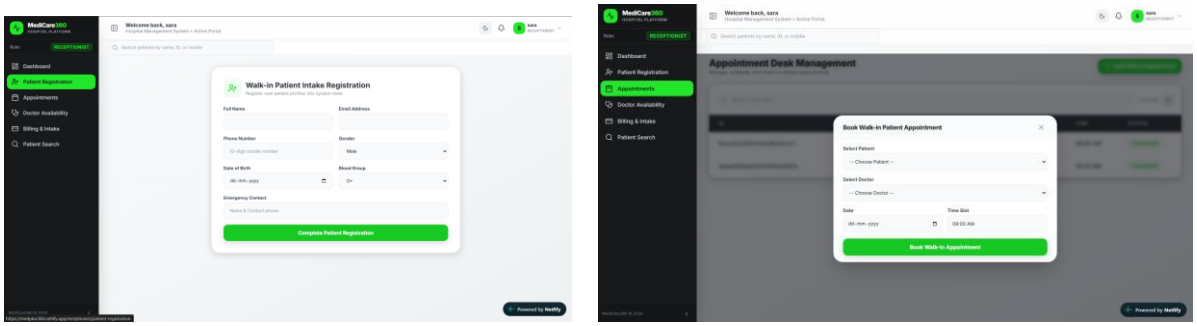


Figure 18: Receptionist Book Walk-in Patient Appointment

6.7 Pharmacist Module Results

The Pharmacist Module is designed to provide pharmacists with access to pharmacy-related information and authorized application functions.

- Secure pharmacist login.
- Pharmacist-specific dashboard.
- Pharmacist profile management.
- Pharmacy department assignment.
- Access to authorized pharmacy information.
- Support for pharmacy-related workflows.
- Role-based access to relevant features.

Administrators can create pharmacist accounts and assign appropriate pharmacy sections, such as Inpatient Pharmacy, Outpatient Pharmacy, and Clinical Pharmacy, if supported by the application.

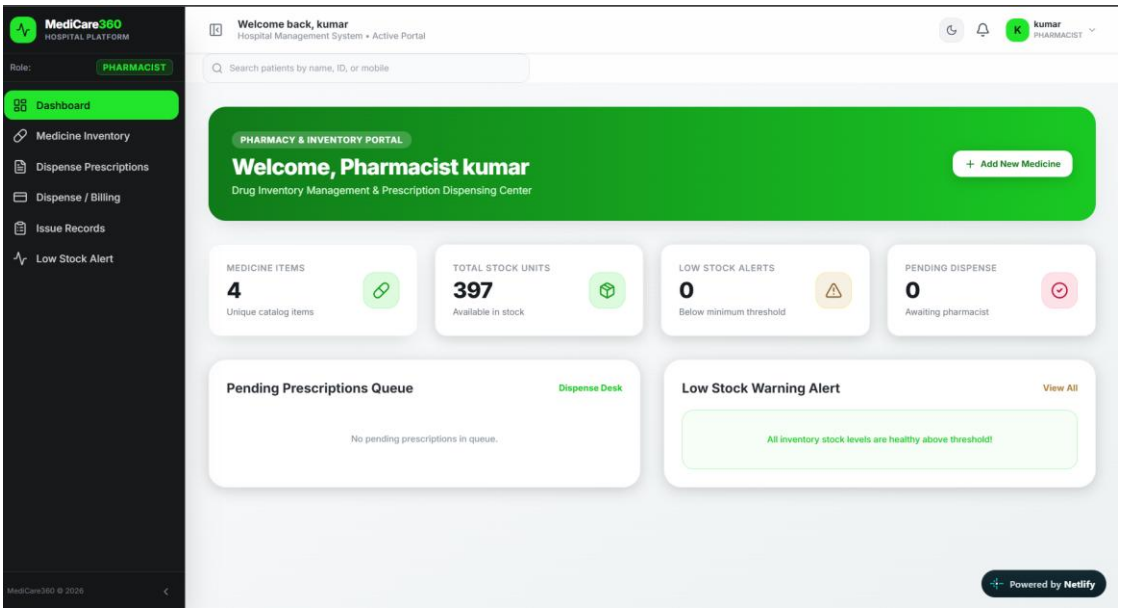


Figure 19: Pharmacist Dashboard

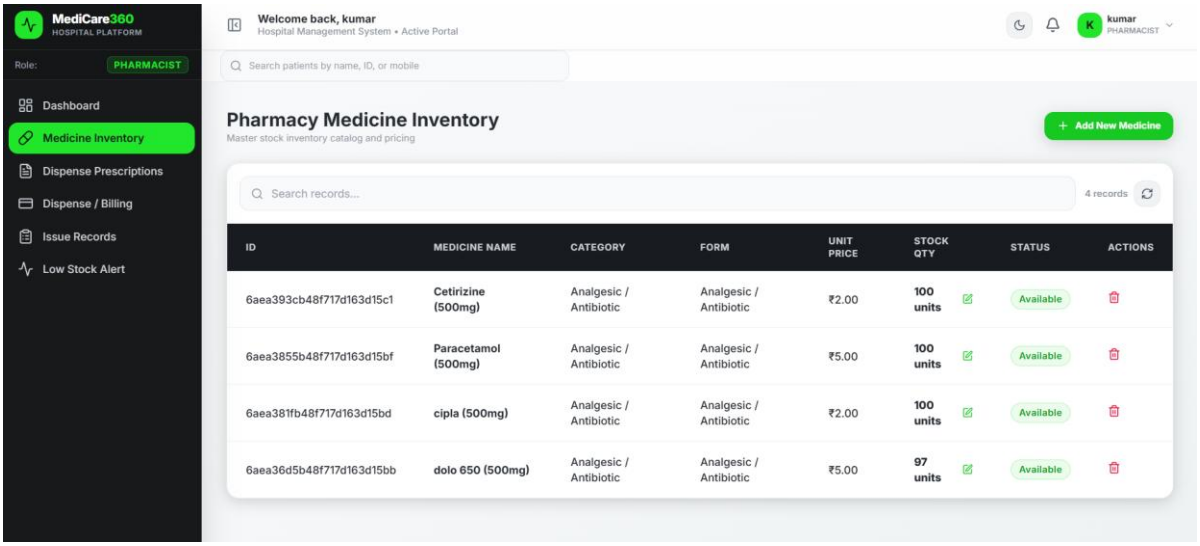


Figure 20: Pharmacy Medicine Inventory

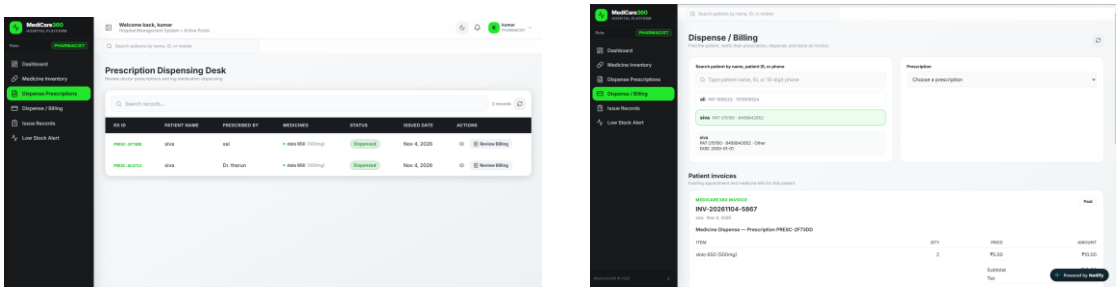


Figure 21 : Prescription Dispensing Desk and Dispense / Billing

6.8 Laboratory Technician Module Results

The Laboratory Technician Module provides laboratory staff with access to authorized laboratory-related information and activities.

- Secure laboratory technician login.
- Laboratory-specific dashboard.
- Laboratory staff profile management.
- Laboratory department and section assignment.
- Access to authorized laboratory information.
- Support for laboratory-related workflows.
- Role-based access to permitted application features.

Administrators can create laboratory technician accounts and assign appropriate laboratory sections, such as Hematology, Biochemistry, Microbiology, and Pathology, depending on the available department configuration.

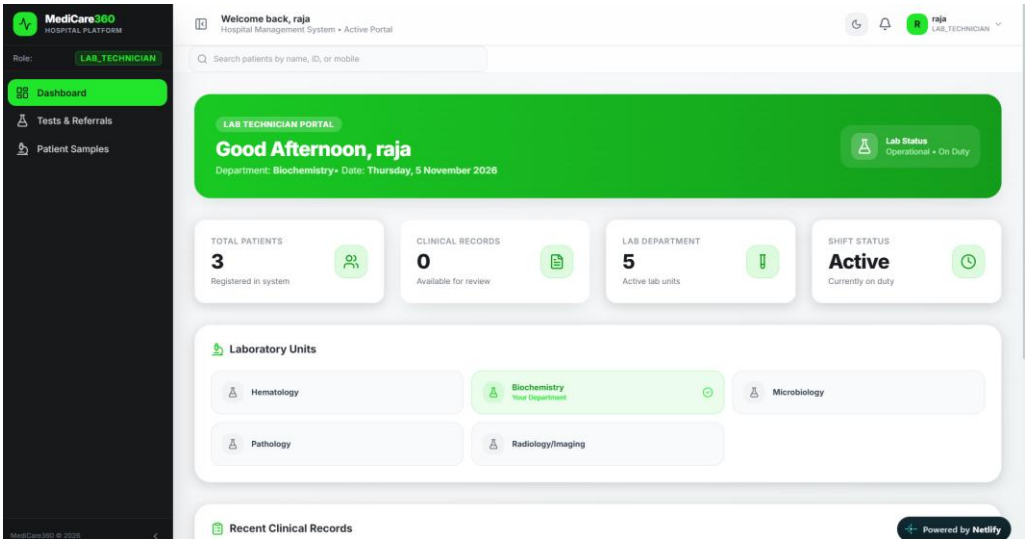


Figure 22: Laboratory Technician Dashboard

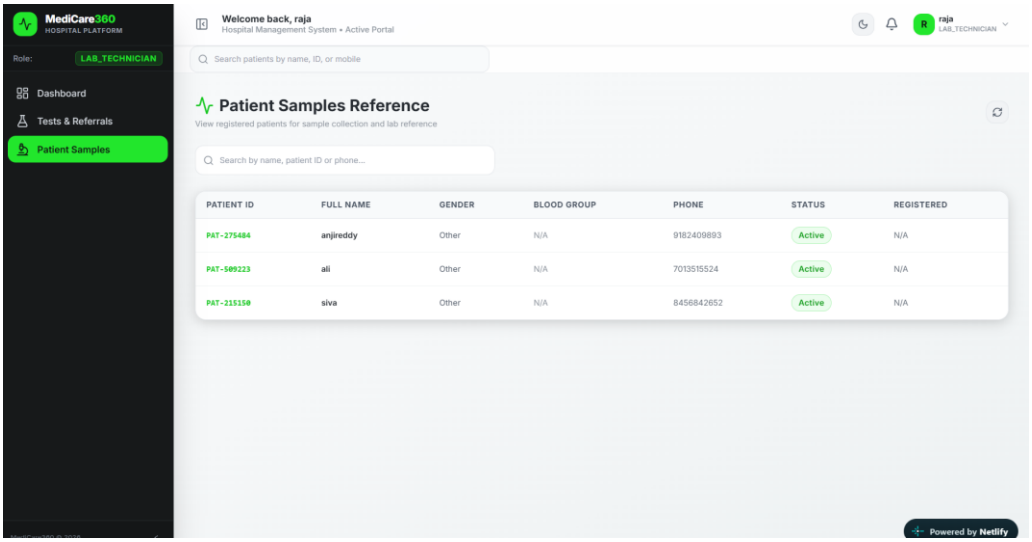


Figure 23: Laboratory Technician in Patient Samples Reference

6.9 Overall Staff Module Result

The Staff Module of MediCare-360 provides a centralized approach to managing different categories of hospital employees. Each staff member receives a separate account based on the role assigned by the administrator.

The role-based authentication system helps prevent unauthorized access between different staff modules. Department and designation assignments help organize employees according to their responsibilities.

Staff account information is maintained through the backend and stored in MongoDB Atlas. This provides persistent storage and allows authorized users to retrieve relevant information when required.

Overall, the Staff Module improves staff account organization, role-based access management, and coordination of hospital-related activities.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENT

7.1 Conclusion

MediCare-360 is developed as a web-based Hospital Management System to simplify hospital administration and improve the organization of patient and staff information.

The application provides a centralized platform with separate access for administrators, patients, doctors, nurses, receptionists, pharmacists, and laboratory technicians.

The system supports administrator-controlled staff account creation, separate patient registration, role-based authentication, department assignment, and centralized data management.

MongoDB Atlas provides persistent storage for application records, while the backend handles authentication, validation, and database communication. The role-based access structure helps ensure that users can access only the features permitted for their responsibilities.

The project demonstrates how web technologies and centralized database management can be used to improve hospital administration, reduce manual work, and organize healthcare-related information.

Overall, MediCare-360 provides a foundation for a more efficient and organized digital hospital management environment.

7.2 Future Enhancements

The following features can be added or improved in future versions of MediCare-360:

1. Online Appointment Management: Introduce advanced appointment booking, scheduling, and availability management for doctors and patients.

2. Electronic Medical Records: Provide a structured system for maintaining patient medical histories, consultation details, and treatment records.

3. Pharmacy Management: Improve medicine inventory tracking, stock management, and prescription-related workflows.

4. Laboratory Management: Introduce laboratory test management, result tracking, and report generation.

5. Billing and Payment Integration: Integrate a secure payment gateway for online consultation fees and hospital billing.

6. SMS and Email Notifications: Send appointment confirmations, account notifications, and important hospital updates.

7. Advanced Reports and Analytics: Generate detailed reports related to patients, staff, appointments, and hospital operations.

8. Mobile Application: Develop Android and iOS applications to improve accessibility for patients and hospital staff.

9. Real-Time Updates: Introduce real-time notifications and status updates for relevant hospital activities.

10. Advanced Security: Implement additional security mechanisms, audit logging, and improved access monitoring.

11. Multi-Hospital Support: Extend the application to support multiple hospitals and their respective departments.

12. AI-Based Healthcare Assistance: Introduce AI-powered features to support administrative analysis, workflow optimization, and healthcare information management.

7.3 Final Summary

MediCare-360 provides a centralized digital solution for hospital management. It aims to simplify administrative activities, organize patient and staff information, and improve coordination between different hospital departments.

The application integrates user management, role-based authentication, separate patient registration, and centralized database storage into one platform.

By using MongoDB Atlas for persistent data management and a modular web application architecture, the system provides a foundation for future improvements and additional hospital management features.

MediCare-360 demonstrates how digital technologies can help hospitals improve information management, reduce manual administrative work, and provide a more organized environment for patients and healthcare professionals.

CHAPTER 8
REFERENCES

8. REFERENCES

1. Ministry of Health and Family Welfare, Government of India, *Electronic Health Record (EHR) Standards for India – 2016*, Government of India, 2016.
2. Ministry of Health and Family Welfare, Government of India, *National Digital Health Blueprint*, Government of India, 2019.
3. National Health Authority, Government of India, *Ayushman Bharat Digital Mission (ABDM)*, Government of India.
4. Ministry of Health and Family Welfare, Government of India, *Health Data Management Policy of Ayushman Bharat Digital Mission*, Government of India, 2020.
5. National Informatics Centre (NIC), Government of India, *eHospital – Hospital Management Information System*, Ministry of Electronics and Information Technology.
6. Ministry of Electronics and Information Technology, Government of India, *Digital Personal Data Protection Act, 2023*, Act No. 22 of 2023.
7. Press Information Bureau, Government of India, *Electronic Health Records*, Ministry of Health and Family Welfare, 5 January 2018.
8. National Health Mission, Government of Tamil Nadu, *Health Management Information System (HMIS)*, Department of Health and Family Welfare, Government of Tamil Nadu.
9. National Health Authority, Government of India, *Healthcare Professionals Registry (HPR) and Health Facility Registry (HFR)*, Ayushman Bharat Digital Mission.
10. Press Information Bureau, Government of India, *National Digital Health Ecosystem*, Ministry of Health and Family Welfare, 25 July 2023.

8.1 Official reference links

S.No	Reference	Official website
1	EHR Standards for India	Download EHR Standards
2	National Digital Health Blueprint	View NDHB
3	Ayushman Bharat Digital Mission	ABDM Portal
4	ABDM Health Data Management Policy	View Policy
5	NIC eHospital	View eHospital
6	Digital Personal Data Protection Act, 2023	View Act
7	Electronic Health Records – Government of India	View Government Reference
8	Tamil Nadu HMIS	View HMIS
9	Healthcare Professionals Registry	View HPR
10	National Digital Health Ecosystem	View Government Report

8.2 references relate to MediCare-360

MediCare-360 module	Relevant references
Hospital management and administration	5, 8
Patient registration and records	1, 3, 5
Doctor and staff management	3, 9
Role-based access and data security	4, 6, 10
MongoDB and digital data management	1, 2, 4
Healthcare information exchange	1, 2, 3